

CT853: ALGORITHMS & LOGICAL FOUNDATIONS

ASSIGNMENT TEMPLATE

NAME: Cian Mac Liatháin

STUDENT NUMBER: 11476078

QUESTION 1:

MARK

Bubble Sort is an example of an exchange sorting algorithm, where successive pairs of adjacent elements are compared and exchanged, if needed, until the entire collection of elements is sorted (Knuth, 1998).

Algorithm 1 BubbleSort

Input: array A of n comparable elements

Output: array A' , a permutation of A where $A'[0] \leq A'[1] \leq \dots \leq A'[n-1]$

Begin

```
1: for  $i = 0$  to  $n - 2$  do
2:   for  $j = n - 1$  to  $i$  do
3:     if  $A[j] < A[j - 1]$  then
4:       exchange  $A[j]$  with  $A[j - 1]$ 
5:     end if
6:   end for
7: end for
```

End

Pseudocode for Bubblesort, written in LaTeX using the using the algorithmicx package (János, 2005).

The analysis of an algorithm can be broken into two subproblems: its correctness, and its complexity (Sedgewick & Wayne, 2014).

Correctness

A correct algorithm is one that is guaranteed to find a solution if a solution exists. In the case of this Bubble Sort, if it is correct then it will always sort an array of comparable elements in ascending order (this can be changed to descending order by swapping the $<$ symbol in the if statement for a $>$ symbol).

Cormen et al. (2009, p.19) advise that we can prove the correctness of an algorithm by studying its loop invariant, the properties that hold true before and after each iteration. There are three aspects of a loop invariant that should be considered:

Initialisation: that it's true prior to the first iteration of the loop.

Maintenance: that if it's true before an iteration of the loop, it stays true before the next iteration.

Termination: that when the loop finishes, there is a useful property that helps to show the correctness of the algorithm.

As there are two loops used in the Bubble Sort pseudocode above, we will show the algorithm's completeness by first investigating the inner loop and using our conclusions there to help investigate the outer loop and the overall algorithm.

a) Inner Loop

Loop invariant: before any given iteration of the inner loop, the minimum element of the subarray $A[i \dots n - 1]$ is found within $A[i \dots j]$.

Initialisation: before the loop's first iteration, $j = n - 1$. It is trivially true that the minimum element of the subarray $A[i \dots n - 1]$ is found within $A[i \dots n - 1]$

Maintenance: depending on where in the array the minimum element is located, there are three different outcomes from each iteration of the loop:

- i. If the minimum valued element of the subarray $A[i \dots n - 1]$ is found within $A[i \dots j - 2]$, outside the range of the inner loop's check between $A[j]$ and $A[j - 1]$, the exchange step does not effect the minimum valued element, leaving it to still be found within $A[i \dots j - 2]$ after the iteration.
- ii. If the minimum valued element of the subarray $A[i \dots n - 1]$ is found at $A[j - 1]$. Then $A[j] > A[j - 1]$ and no exchange occurs, leaving the minimum valued element to still be found within $A[i \dots j - 1]$ after the iteration.
- iii. If the minimum valued element of the subarray $A[i \dots n - 1]$ is found at $A[j]$. Then the if statement $A[j] < A[j - 1]$ is true, and the elements are exchanged leaving the minimum valued element to be found within $A[i \dots j - 1]$. In the case of both $A[j - 1]$ and $A[j]$ holding the same minimum valued element an exchange does not occur, but then the minimum value is already still found within $A[i \dots j - 1]$.

In all of these scenarios it is shown that after each iteration the minimum value of the subarray $A[i \dots n - 1]$ is left to be found within $A[i \dots j - 1]$. When j is decremented in the next iteration, the loop invariant still holds as the minimum valued element will be found within $A[i \dots j]$.

Termination: the inner loop terminates when j is 1 value above that of i , meaning that due to the exchange of $A[j]$ and $A[j - 1]$ if $A[j] < A[j - 1]$ the minimum value of the subarray $A[i \dots n - 1]$ is found at $A[j - 1] == A[i]$.

b) Outer Loop

Loop invariant: the smallest i elements are found in the first i indices of the array, in sorted order.

Initialisation: before the first iteration, $i = 0$. The smallest 0 elements are in the first 0 indices of the array, which is trivially true.

Maintenance: if we suppose that the smallest i elements are found in the first i indices in sorted order, $A[0 \dots i - 1]$, then we can see that these will never be changed by the inner loop as it only ever exchanges the values $A[j]$ and $A[j - 1]$ and is constrained $j > i$. From our analysis of the inner loop variant, we see that after each iteration of the inner for loop the smallest valued element of $A[i \dots n - 1]$ is found at $A[i]$. As i increments the next smallest element will be left in the place of the array leading to the i smallest elements being found in the first i indices of the array, in sorted order.

Termination: at the end of the outer for loop, $i = n - 2$. So the smallest $n - 2$ elements are found in the first $n - 2$ places, in sorted order. This means also that the element found at index $n - 1$ is the smallest element, aka the maximum element, leaving the entire array sorted.

We have now shown, by studying the inner and outer loop variants, that the algorithm terminates with a permutation of the original array A sorted in such a way that $A'[0] \leq A'[1] \leq \dots \leq A'[n - 1]$. The algorithm has been shown to be complete.

Complexity

There are many external factors that can impact the efficiency of an algorithm, such as the programming language or hardware used. In order to consider and compare algorithms in a language and hardware agnostic way, computer scientists often identify the resources required to run an algorithm by studying the algorithm's computational complexity. Time complexity and space complexity are the two most commonly considered. (Sedgewick & Wayne, 2014)

An algorithm's computational complexity can be considered as a function of input size, and this function can be analysed further through the use of asymptotic notation. One example of which is Big O Notation, which classifies algorithms based on the asymptotic upper bound of their growth rates. Other asymptotic notations include Little-o notation o and Big Omega notation Ω , which classify based on the loose upper bound and the tight lower bound of an algorithm's growth rate, respectively. The two most common complexities considered when analysing algorithms are time (the time it takes for the algorithm to run/complete) and space complexity (the memory used during the running of the algorithm).

Time Complexity of Bubble Sort

We can see that when $i = 0$ the inner for-loop makes $n - 1$ comparisons. When $i = 1$, it makes $n - 2$ comparisons and so on until $i = n - 2$ where only 1 comparison is made. Using the formula for an arithmetic series $\sum_{q=1}^p q = \frac{p(p+1)}{2}$, the total sum of these comparisons is then

$$(n - 1) + (n - 2) + \dots + 2 + 1 = \frac{(n - 1)(n - 1 + 1)}{2} \\ \frac{n^2 - n}{2}$$

We can see the most significant factor of growth here is the n^2 term, meaning that the Big O notation of Bubble Sort's time complexity is $O(n^2)$. That is that the runtime grows quadratically depending on the size of the input array being sorted, with the current implementation of Bubble Sort shown in the pseudocode the algorithm would have this same running time in all cases. It could be improved, however, by adding a check whether any swaps have actually taken place during the inner-loop's sweep of the array, if no swaps have occurred then the array is already in sorted order and the algorithm can terminate. This would give Bubble Sort a best-case running time of $O(n)$ as the outer loop would break after the inner-loop's first $n - 1$ comparisons.

```
54     public static void improvedBubbleSort(float[] numbers) {
55         boolean swapped = false;
56         for (int i = 0; i < numbers.length - 2; i++) {
57             for (int j = numbers.length - 1; j > i; j--) {
58                 if (numbers[j] < numbers[j - 1]) {
59                     float temp = numbers[j];
60                     numbers[j] = numbers[j - 1];
61                     numbers[j - 1] = temp;
62                     swapped = true;
63                 }
64             }
65             if (!swapped) {
66                 break;
67             }
68         }
69     }
```

Space Complexity of Bubble Sort

Bubble Sort is an example of an in-place sorting algorithm, the space it uses during its runtime is not dependent on the input size as it is limited to a constant number of variables such as counters. Bubble Sort has a space complexity of $O(1)$

QUESTION 2:**MARK**

MergeSort is an example of a divide and conquer sorting algorithm, where the problem space is first partitioned into successively smaller groups, comparisons are made traversing these smaller groups rather than the original, and these subgroups are combined back to the size of the original. (Skiena, 2020)

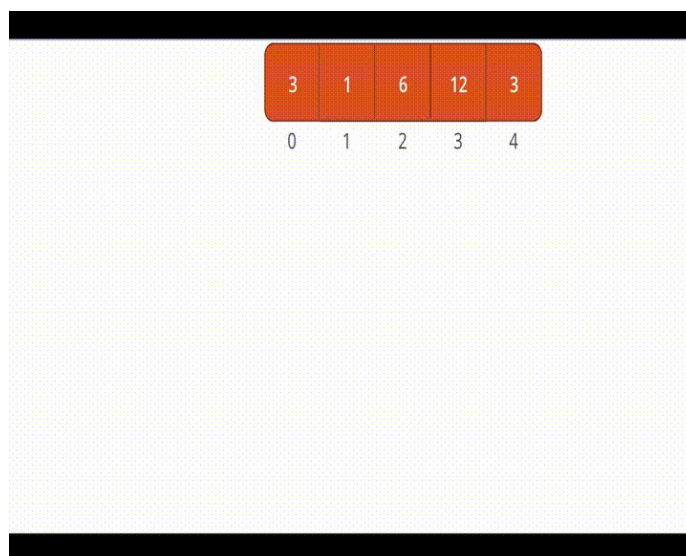
Cormen et al. (2009, p.30) give an outline of how the MergeSort algorithm follows the divide-and-conquer paradigm:

Divide: Divide an array of n comparable elements into two subarrays of length $\frac{n}{2}$

Conquer: Sort these subarrays recursively using MergeSort

Combine: Merge the two sorted subarrays to produce a sorted array of size n

We can see from the pseudocode below that while the left index is smaller than the right index, a new index m is made half-way between l and r . MergeSort is then called recursively on two subarrays that range from l to m and $m + 1$ to r . This eventually leads to n arrays with a single element each, meaning they are trivially sorted. These arrays are then merged by comparing each single element in adjacent arrays and placing them, sorted, in an array of size 2. These arrays are then merged further by comparing the first element of each, then the second element, into an array of size 4. This continues until all elements are sorted into an array of size n . This process is visualised below:



Visualisation of the partitioning and merging of the mergeSort algorithm, based on visualiser from HackerEarth (2022)

Algorithm 2.1 MergeSort

Input: array A of n comparable elements and array indices l and r where $l \leq r$

Output: array A' , a permutation of A where $A'[l] \leq \dots \leq A'[r]$

Begin

- 1: **if** $l < r$ **then**
- 2: $m = l + \frac{r-l}{2}$
- 3: $\text{mergeSort}(A, l, m)$
- 4: $\text{mergeSort}(A, m + 1, r)$
- 5: $\text{merge}(A, l, m, r)$
- 6: **end if**

End

Algorithm 2.2 Merge

Input: array A of n comparable elements, array indices p , q and r

Output: array A' , a permutation of A where $A'[p] \leq \dots \leq A'[q] \leq \dots \leq A'[r]$

Begin

```
1:  $n1 = q - p + 1$ 
2:  $n2 = r - q$ 
3: create arrays  $L[0\dots n1]$  and  $R[0\dots n2]$ 
4: for  $i = 0$  to  $n1$  do
5:    $L[i] = A[p + i]$ 
6: end for
7: for  $j = 0$  to  $n2 - 1$  do
8:    $R[j] = A[q + 1 + j]$ 
9: end for
10:  $L[n1] = \text{inf}$ 
11:  $R[n2] = \text{inf}$ 
12:  $i = 0$ 
13:  $j = 0$ 
14: for  $k = p$  to  $r$  do
15:   if  $L[i] \leq R[j]$  then
16:      $A[k] = L[i]$ 
17:      $i = i + 1$ 
18:   else
19:      $A[k] = R[j]$ 
20:      $j = j + 1$ 
21:   end if
22: end for
```

End

Pseudocode for MergeSort and Merge, written in LaTeX using the using the algorithmicx package (János, 2005).

Correctness of MergeSort (Algorithm 2.1)

We need to show that MergeSort correctly sorts any array of size n where $n \geq 1$. This can be done through induction:

When the indices l and r are chosen to be the first and last index of the array, we can see that $r - l + 1 = n$

In an array of size $n = 1$, the indices l and r must be equal. MergeSort's if statement returns false and no more work is done, as an array with a single element is already sorted.

In an array of size $n \geq 2$, it is true that $r \geq l$ and MergeSort calls itself recursively and the Merge algorithm. Due to the calculation of $m = l + \frac{r-l}{2}$, which will always be smaller than r , and the use of m as the end index at line 3 and $m + 1$ as the starting index at line 4, the recursive MergeSort calls are of a smaller sized n , eventually leading to the proven case where $n = 1$. The overall algorithm's correctness will then depend then on showing that the $\text{merge}(A, l, m, r)$ call of line 5 is also correct.

Correctness of Merge (Algorithm 2.2)

Although there are other loops used to make temporary arrays, the core loop of the Merge algorithm is found at line 14 of the pseudocode.

Loop Invariant: before any given iteration of the loop, the output array $A'[p \dots k]$ contains the $k - p$ smallest elements of L and R , in sorted order. The smallest elements of L and R not already copied to A' are found at $L[i]$ and $R[j]$, respectively.

Initialisation:

before the first iteration of the loop $k = p$, $i = 0$ and $j = 0$, it is trivially true that the array A' contains the 0 smallest elements of L and R in sorted order. The smallest uncopied elements of L and R are found at $L[0]$ and $R[0]$.

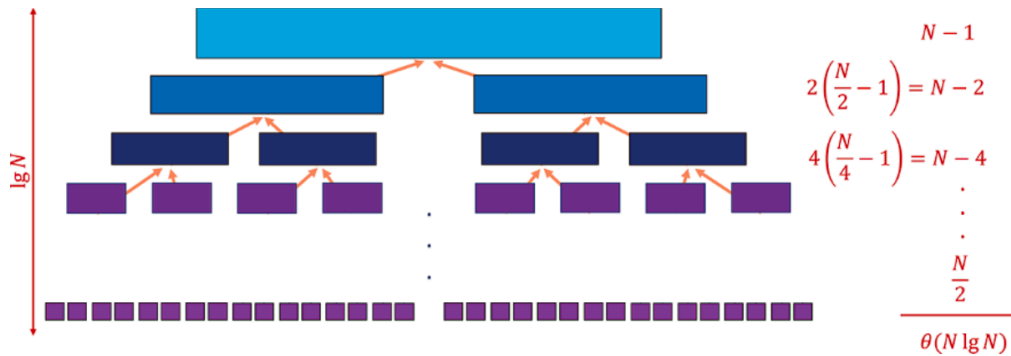
Maintenance:

in the case the $L[i] \leq R[j]$ then $L[i]$ is the next smallest element to be copied to the array, which already holds the first $k - p$ smallest elements. This element is copied to $A'[k]$ and i is incremented, meaning that at the beginning of the next iteration of the while loop the smallest element of L not already copied to A' is still found at $L[i]$. We can see that if $L[i] > R[j]$ the else statement carries out the same copying and incrementing procedure in terms of $R[j]$. k is then incremented in order for the invariant of A' holding the $k - p$ smallest elements to stay true.

Termination:

the loop terminates when $k = r + 1$. The permutation array A' then contains the $(r + 1) - p = r - p + 1$ elements sorted in increasing order as required by the output $A'[p] \leq \dots \leq A'[r]$

Time Complexity of Merge Sort



Balanced recursion tree for MergeSort from Skiena (2020, p.128)

The partitioning aspect is visualised in the depth of the recursion tree above, as the array is repeatedly halved this leads to a depth of $\log_2 n$. The work at each level involves the comparison and merging of adjacent arrays. These arrays are already sorted in increasing order, which means that the smallest element of both arrays must sit at the beginning of the two arrays. When this smallest is chosen and copied to the next combined array, the two arrays are still left sorted in increasing order but one slightly smaller than the other. The next smallest element must now again be at the beginning of one of these two arrays. When this operation is repeated until both compared arrays are empty, with a total of n elements in both arrays overall, at most $n - 1$ comparisons leading to an $O(n)$ of work. The overall time complexity is then $\theta(n * \log(n)) = O(n * \log(n))$.

Space Complexity of Merge Sort

The creation of temporary arrays during each recursive call to MergeSort (and thus to Merge) lead to a space complexity of $O(n)$.

QUESTION 3:	MARK
Below are pseudocode outlining the iterative and recursive implementations of a Fibonacci algorithm, used to compute the nth number of the famous Fibonacci series: Algorithm 3.1 Fibonacci (Iterative) Input: a non-negative integer n Output: the nth number of the Fibonacci series Begin <pre> 1: if $n \leq 1$ then return n 2: end if 3: $x = 0$ 4: $y = 1$ 5: for $i = 2$ to n do 6: $z = x + y$ 7: $x = y$ 8: $y = z$ 9: end for 10: return $x + y$ </pre> End Algorithm 3.2 Fibonacci (Recursive) Input: a non-negative integer n Output: the nth number of the Fibonacci series Begin <pre> 1: if $n \leq 1$ then return n 2: else return $Fibonacci(n - 1) + Fibonacci(n - 2)$ 3: end if </pre> End Pseudocode for the Fibonacci algorithms, written in LaTeX using the algorithmicx package (János, 2005). Correctness The correctness of the algorithm can be proven by induction. When $n \leq 1$ the algorithm returns n itself, which is the correct result. For now, we assume that the algorithm correctly calculates the Fibonacci numbers $n \leq k$ where $k > 1$. All Fibonacci numbers after the first 2 terms are calculated as the sum of the previous 2 terms. In the case of $n = k + 1$: $Fibonacci(k + 1) = Fibonacci(k) + Fibonacci(k - 1)$ As we have already shown the correctness for when $n \leq 1$, and these earlier terms are summed to make successive terms, we can see that both of our implementations of Fibonacci above are correct as that is what occurs in Algorithm 3.1's for loops and Algorithm 3.2's else return statement.	

The upper limit of the number of function calls made by the recursive Fibonacci algorithm is shown to be $\approx 2^n - 1$, leading to an exponential time complexity $O(2^n)$. The depth of this recursion tree, the algorithm's space complexity, is $O(n)$.

In contrast with the recursive Algorithm 3.2, the iterative alternative does not make repeated calls to functions previously calculated, and instead stores the required values for later addition. The for loop from lines 5-9 calculate the the latest Fibonacci number z as the sum of the two previous numbers and updates those previous numbers in advance of the next iteration of the for loop. This minimises any unnecessary computation as each of the values required to evaluate the specified n th Fibonacci number is calculated sequentially as the sum of two integers, leading to a time complexity of $O(n)$. The variables/space used remain constant regardless of input size, leading to a space complexity of $O(1)$

QUESTION 4:

MARK

Exponentiation by squaring is another example of a divide-and-conquer algorithm, and it is used to lower the computation required to exponentiate an integer in comparison to a naïve, iterative approach. A naive approach is shown below:

```
58 public static float expBySquaringNaive(float base, float exponent) {
59
60     if (exponent < 0) {
61         base = 1 / base;
62         exponent = -exponent;
63     }
64
65     float result = base;
66     for (int i = 2; i <= exponent; i++) {
67         result = result * base;
68     }
69     return result;
70 }
```

We can see from this code that for an exponent n , the base number is multiplied by itself $n - 1$ times. Luckily, the laws of indices are well-defined, and these can be taken advantage of to lower the computation of exponentiation. In particular, the rules, $x^{p+q} = x^p x^q$ and $(x^p)^q = x^{pq}$ mean that any odd exponent can be broken down into an exponent of one multiplied by an even exponent e.g. $x^9 = x(x^8)$, and even exponents can be halved by squaring e.g. $x^8 = (x^2)^4$. Furthermore, the rules $x^0 = 1$, $x^1 = x$, $x^{-p} = \frac{1}{x^p}$ can be used to produce the following pseudocode to covering all cases of exponents:

Algorithm 4 Exponentiation by Squaring (Recursive)

Input: integers x and n , where x is a base number and n its exponent

Output: the result of x^n

Begin

- 1: **if** $n = 0$ **then return** 1
- 2: **else if** $n = 1$ **then return** x
- 3: **else if** $n < 0$ **then return** $\text{expBySquaring}(\frac{1}{x}, -n)$
- 4: **else if** n is even **then return** $\text{expBySquaring}(x * x, n/2)$
- 5: **else if** n is odd **then return** $x * \text{expBySquaring}(x * x, (n - 1)/2)$
- 6: **end if**

End

Pseudocode for Exponentiation by Squaring (Recursive), written in LaTeX using the algorithmicx package (János, 2005).

We can see from this pseudocode that when $n \geq 2$, the base number is squared and the exponent is halved in the next recursive call, until the exponent is lowered to a 1 or 0. In the case of an odd exponent, the laws of indices are used to multiply x^1 by the recursive call and lowering the exponent by 1, making it an even exponent in the next function call. This arithmetic in the return call when n is odd, however, means that the algorithm above is not tail-recursive. A tail-recursive algorithm's return call should only be a recursive call to itself, with no arithmetic or extra computation to be made on the result of that call (Skiena, 2020).

This is beneficial as it eliminates the need for the stack space to store the function arguments to be used in other calculations. We can see from the pseudocode above that for $n \geq 2$ the exponent is halved each time, leading to a time complexity of $O(\log n)$ compared to $O(n)$ of the naive implementation.

Using the example of 2^{20} , the result of the Java implementations of both algorithms are shown below:

```
Please enter the base number: 2
Please enter the exponent: 20
Naive Result: 1048576.0
Runtime of Naive Implementation: 18.133407ms
Tail-Recursive Result: 1048576.0
Runtime of Tail-Recursive Implementation: 0.244858ms
Exponentiation-By-Squaring was 17.888548999999998ms faster
```

The running time of each algorithm is calculated by calls to the `System.nanoTime()` before and after the call to each algorithm. We can see that the recursive algorithm is much faster in practice, as expected.

Reference List

- Cormen, T. H., Leiserson, C. E., Rivest, L. R., Stein, C. (2009) *Introduction to Algorithms*. 3rd edn. Massachusetts, USA: The MIT Press.
- János, S. (2005) *LaTeX AlgorithmicX Package*. Available at:
<https://texdoc.org/serve/algorithmicx/0>
- HackerEarth (2022) *Algorithm Visualiser - MergeSort*. Available at:
<https://www.hackerearth.com/practice/algorithms/sorting/merge-sort/visualize/>
- Knuth, D. (1998) *The Art of Computer Programming*. 3rd edn. Massachusetts, USA: Addison-Wesley.
- Sedgewick, R. & Wayne, K. (2014) *Algorithms*. 4th edn. NJ, USA: Addison-Wesley.
- Skiena, S. S. (2020) *The Algorithm Design Manual*. 3rd edn. Cham, Switzerland: Springer International Publishing.